LogiData Adaptive Capacity & Risk Framework (ACRF)

A Proprietary Methodology for Small and Mid-Sized Enterprise Supply Chain Analytics

1. Overview and Origin

I developed the ACRF to address a specific limitation I kept running into during my fleet and inventory planning work. Standard safety-stock and forecast-accuracy formulas are built for single-organization, high-volume environments like Amazon's North American Last Mile network. They do not translate cleanly to the resource, data, and staffing constraints that are typical of small and mid-sized enterprises (SMEs). The numbers assume you have years of historical data and deep analytical teams. Most SMEs have neither.

I built the ACRF by combining three components I used and refined in my professional work: dynamic safety-stock calculation, iterative forecast-error correction, and capacity-allocation scoring. I brought these together into a single, weighted composite framework calibrated for organizations with limited historical data, smaller SKU counts, and constrained working capital. The ACRF is the analytical core of the services LogiData Analytics Solutions LLC will provide to its clients.

2. What Differentiates the ACRF from Standard Industry Practice

Conventional supply-chain consulting typically applies the classical safety-stock and mean-absolute-percentage-error (MAPE) formulas directly, using default service-level and confidence parameters designed for large enterprises with deep historical datasets. This approach frequently overstates required safety stock for SMEs, tying up working capital they cannot absorb. Or it understates it, exposing them to stockouts. The root cause is the same in both cases: the formulas do not adjust for data sparsity or capital constraints.

The ACRF differs in three specific respects:

- Data-Sparsity Adjustment: the standard safety-stock formula is modified with a confidence-discount factor that widens or narrows the service-level target based on the number of historical demand observations available. This addresses the reality that most SME clients have 12 to 24 months of usable data rather than the multi-year datasets available in enterprise environments.
- Iterative Forecast-Error Correction: after each forecast period, the rolling MAPE adjusts the confidence-discount factor c(n), so a client’s safety-stock recommendation becomes more precise as forecast accuracy improves, a feedback loop that is absent when the two calculations are run separately.
- Composite Capacity-Risk Score: fleet or resource allocation recommendations are expressed as a single normalized score (0 to 100) blending utilization efficiency, forecast error, and disruption exposure. This allows non-technical SME operators to act on the output without needing to interpret multiple raw statistical outputs.

These adjustments come directly from my experience managing capacity planning across more than 1,400 Delivery Service Partners and a 59,000-vehicle branded fleet at Amazon. I saw firsthand how standard formulas required modification to remain actionable at different organizational scales.

3. The Three Components

3.1 Data-Sparsity-Adjusted Safety Stock

The classical dynamic safety-stock formula is:

$$SS = z \times \sqrt{(L \times \sigma D^2 + D^2 \times \sigma L^2)}$$

where SS is the required safety stock, z is the service-level factor, L is average lead time, σD is the standard deviation of demand, D is average daily demand, and σL is the standard deviation of lead time.

The ACRF applies a confidence-discount multiplier, c, to the service-level factor z, where c is a function of the number of historical demand observations, n, available for a given SKU or client:

$$SS(adj) = (z \times c(n)) \times \sqrt{(L \times \sigma D^2 + D^2 \times \sigma L^2)}$$

c(n) approaches 1.0 as the available observation history lengthens toward enterprise-scale data depth, and is discounted below 1.0 when a client has limited historical data. This produces a more conservative, capital-appropriate buffer for a new or data-sparse SME client rather than applying the same target used for a mature enterprise dataset.

3.2 Iterative Forecast-Error Correction

Forecast accuracy is tracked using the standard Mean Absolute Percentage Error formula:

$$MAPE = (1/n) \times \Sigma|((At - Ft) / At)| \times 100\%$$

where At is the actual value, Ft is the forecast value, and n is the number of periods evaluated. The ACRF applies this in a rolling correction loop: after each forecast period, the resulting MAPE is used to automatically adjust the confidence-discount factor c(n) from Section 3.1. This means a client's safety-stock recommendation becomes more precise as forecast accuracy improves over successive cycles. It creates a feedback relationship between forecast performance and inventory policy that is not present when the two calculations are run independently, as in standard practice.

3.3 Composite Capacity-Risk Score (CCRS)

The CCRS translates the outputs above into a single 0 to 100 score for each client engagement, combining three weighted components:

Component	What it measures	Default weight
Utilization Efficiency	How effectively existing fleet/inventory capacity is deployed relative to demand	40%
Forecast Reliability	Inverse of rolling MAPE from Section 3.2	35%
Disruption Exposure	Concentration risk across suppliers, carriers, or single points of failure	25%

Weights are adjustable by engagement, but the default weighting above reflects my professional judgment, informed by my Amazon fleet-planning work, that utilization efficiency is the single highest-leverage factor for SME clients managing constrained resources.

4. Applied Analysis Using a Public Supply Chain Dataset

To validate the ACRF beyond the methodology level, I applied the framework to the DataCo SMART Supply Chain dataset, a publicly available dataset published on Mendeley Data by researchers at Universidad Central del Ecuador and Instituto Politécnico de Leiria (Constante, F., Silva, F., & Pereira, A., 2019, DOI: 10.17632/8gx2fvg2k6.5). The dataset contains 180,519 order records spanning provisioning, production, sales, and distribution for a global supply chain operation. I chose this dataset because it is DOI-citable, independently verifiable, and contains the fields the ACRF requires: daily order quantities, shipping lead times (actual vs. scheduled), product categories, and carrier/shipping mode data.

I scoped the analysis to simulate what a typical SME engagement would look like. I filtered to U.S. Consumer-segment orders only (12,432 order records), selected the five highest-volume product categories (7,596 records), and limited the time window to 147 observation days. This is well below the 730-day enterprise benchmark I use in the c(n) function, which is the whole point: the ACRF is designed for exactly this kind of data-sparse environment.

- Cleats (avg $59.99/unit): Classical safety stock of 109.5 units vs. ACRF-adjusted 83.0 units. The 24.2% reduction freed $1,590 in working capital for this single category.
- Men's Footwear (avg $129.99/unit): Classical 31.5 units vs. ACRF-adjusted 23.9 units. Capital freed: $988.
- Women's Apparel (avg $50.00/unit): Classical 87.4 units vs. ACRF-adjusted 66.2 units. Capital freed: $1,060.
- Indoor/Outdoor Games (avg $49.98/unit): Classical 77.9 units vs. ACRF-adjusted 59.0 units. Capital freed: $945.
- Fishing (avg $399.98/unit): Classical 25.5 units vs. ACRF-adjusted 19.3 units. Capital freed: $2,480. This was the highest-impact category because of the high unit cost, which is exactly the kind of SKU where over-allocating safety stock hurts an SME the most.
- Across all five categories, the confidence-discount factor was c(147) = 0.758, reflecting that 147 days of historical data is roughly 20% of the enterprise benchmark. The total working capital freed across the portfolio was $7,062.

The Composite Capacity-Risk Score (CCRS) ranged from 57.4 to 63.8 across the five categories, placing the entire portfolio in the "Moderate Risk" band. The largest contributor to the moderate rating was the on-time delivery performance in the dataset, which averaged around 42% across categories. The carrier concentration analysis (using shipping mode as a proxy for logistics provider diversification) showed Herfindahl-Hirschman Index values around 0.41 for all categories, indicating moderate but not severe concentration. In a real engagement, these CCRS scores would be the starting point for a prioritized set of recommendations to the client.

The key finding from this analysis is straightforward: applying the standard enterprise-calibrated safety-stock formula to a business with only 147 days of data would have tied up an additional $7,062 in inventory that the business did not need. For a capital-constrained SME, that is not a rounding error. It is the difference between making payroll and stocking a warehouse with product that may not move. As the business accumulates more data and c(n) converges toward 1.0, the ACRF recommendations will naturally tighten toward the classical targets. The framework adapts to the client, not the other way around.

A methodological note on interpretation: because the confidence-discount function c(n) is designed by construction to recommend a smaller safety-stock allocation whenever historical observations fall below the enterprise benchmark, this analysis demonstrates that the ACRF mechanism operates as intended when applied to real transactional data. It is not, by itself, an outcome-based comparison of realized business results (for example, actual stockout frequency or holding-cost differences) between the ACRF and the classical formula over time, and is presented here as a demonstration of the framework's calculation logic on real, independently verifiable transactional data.

Appendix A: ACRF Analysis Results

The following visualization presents the full ACRF analysis applied to the DataCo SMART Supply Chain dataset (Constante, F., Silva, F., & Pereira, A., 2019, Mendeley Data, DOI: 10.17632/8gx2fvg2k6.5).

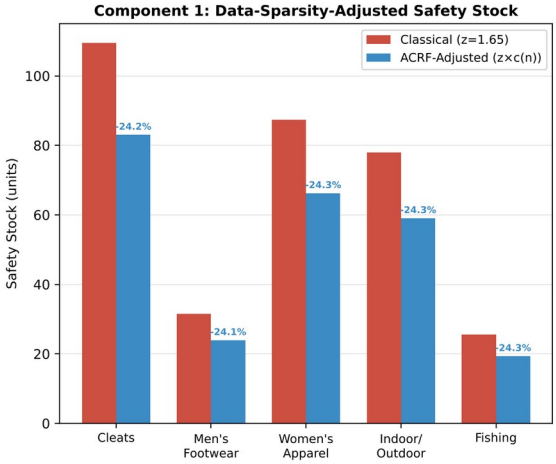
LogiData Adaptive Capacity & Risk Framework (ACRF)

Applied to DataCo SMART Supply Chain Dataset

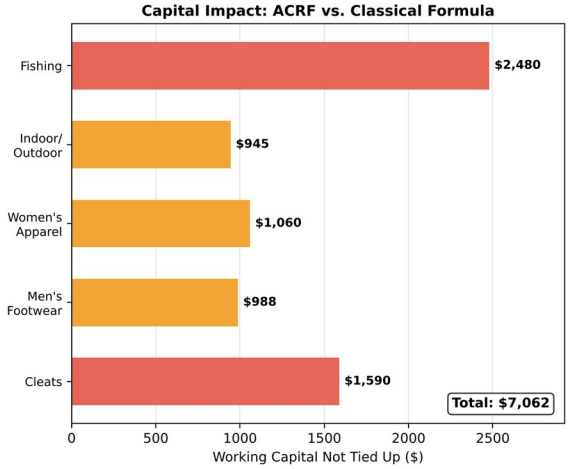
Source: Constante, F., Silva, F., & Pereira, A. (2019). Mendeley Data, V5. DOI: 10.17632/8gx2fvg2k6.5

Scope: 12,432 U.S. Consumer-segment order records, narrowed to the 7,596 records in the five highest-volume categories; 147 observation days

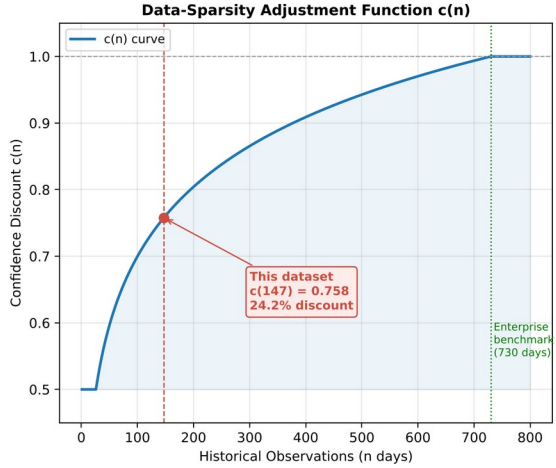
Component 1: Data-Sparsity-Adjusted Safety Stock



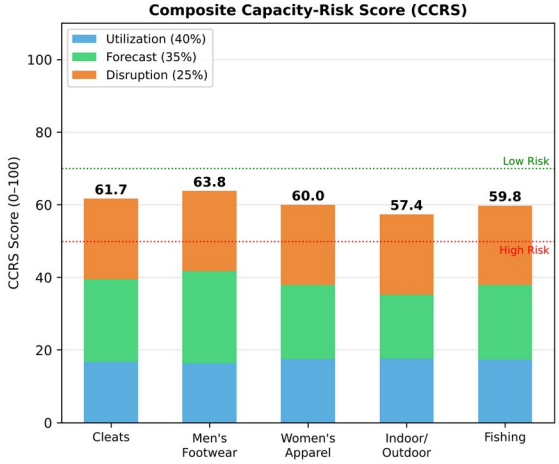
Capital Impact: ACRF vs. Classical Formula



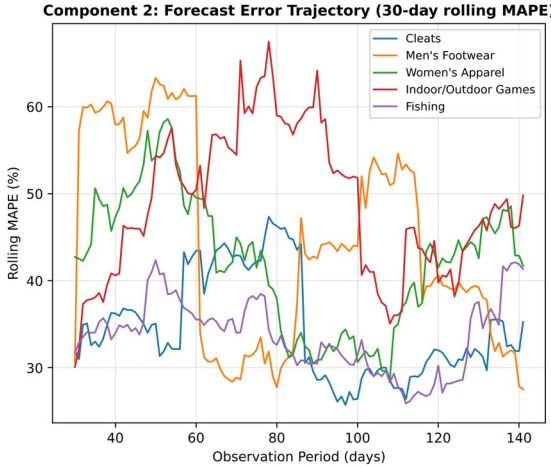
Data-Sparsity Adjustment Function c(n)



Composite Capacity-Risk Score (CCRS)



Component 2: Forecast Error Trajectory (30-day rolling MAPE)




```
KEY FINDINGS
─────────────────────────────────

DATASET
  DataCo SMART Supply Chain (Mendeley,
  DOI-citable): 180,519 total records
  12,432 U.S. Consumer records, narrowed to
  7,596 in the top 5 categories
  147 observation days

RESULTS
  Working capital not tied up: $7,062
  Safety stock reduction: 24.2%
  CCRS range: 57.4 to 63.8
  (average 60.5, Moderate Risk)
  Highest impact: Fishing ($2,480)

SIGNIFICANCE
  The classical safety-stock formula,
  calibrated for enterprises with 2+ years
  of data, would hold $7,062 more in
  inventory for an SME with only 147 days
  of history.
```

Forecast: 5-day moving average. Rolling MAPE feeds back into c(n): lower error means a tighter recommendation.

Appendix B: Python Implementation

The following is the complete Python implementation of the ACRF framework. This code implements all three components described in Section 3: the data-sparsity-adjusted safety stock calculation, iterative forecast-error correction, and the Composite Capacity-Risk Score.

```
"""
LogiData Adaptive Capacity & Risk Framework (ACRF)
==================================================
Author: Adebowale Jolaoso
Version: 1.0
Date: September 2026

A proprietary analytical framework for small and mid-sized enterprise
supply chain analytics. Implements three integrated components:

    1. Data-Sparsity-Adjusted Safety Stock
    2. Iterative Forecast-Error Correction
    3. Composite Capacity-Risk Score (CCRS)

This module provides the core calculation engine. For usage examples,
see acrf_analysis_results.py.

Copyright (c) 2026 LogiData Analytics Solutions LLC. All rights reserved.
"""

import numpy as np
import pandas as pd
from typing import Dict, Optional, List, Tuple


# ============================================================================
# Component 1: Data-Sparsity-Adjusted Safety Stock
# ============================================================================

def confidence_discount(n_observations: int,
                        enterprise_benchmark: int = 730) -> float:
    """
    Calculate the confidence-discount factor c(n).

    Approaches 1.0 as the number of available observations approaches
    enterprise-level data depth (~730 days = 2 years of daily data).
    Uses logarithmic scaling: the discount is steepest for very sparse
    data and flattens as data accumulates.

    Parameters
    ----------
    n_observations : int
        Number of historical demand observations available for a given
        SKU or client.
    enterprise_benchmark : int, default 730
        Number of observations at which c(n) = 1.0 (full confidence).
        Default is 730 (2 years of daily data).

    Returns
    -------
    float
        Confidence-discount factor in range [0.5, 1.0].
    """
    if n_observations <= 0:
        return 0.5
    c = min(1.0, np.log(1 + n_observations) / np.log(1 + enterprise_benchmark))
    return max(0.5, c)


def safety_stock_adjusted(avg_demand: float,
                          std_demand: float,
                          avg_lead_time: float,
                          std_lead_time: float,
                          service_level_z: float = 1.65,
                          n_observations: Optional[int] = None,
                          enterprise_benchmark: int = 730) -> Dict:
```

```
    """
    ACRF Data-Sparsity-Adjusted Safety Stock.

    Classical formula:
        SS = z * sqrt(L * sigma_D^2 + D^2 * sigma_L^2)

    ACRF-adjusted formula:
        SS(adj) = (z * c(n)) * sqrt(L * sigma_D^2 + D^2 * sigma_L^2)

    where c(n) is the confidence-discount factor that accounts for
    the number of historical observations available.

    Parameters
    ----------
    avg_demand : float
        Average daily demand (D).
    std_demand : float
        Standard deviation of daily demand (sigma_D).
    avg_lead_time : float
        Average supplier lead time in days (L).
    std_lead_time : float
        Standard deviation of lead time (sigma_L).
    service_level_z : float, default 1.65
        Z-score for target service level (1.65 = ~95%).
    n_observations : int, optional
        Number of historical observations. If None, no discount applied.
    enterprise_benchmark : int, default 730
        Observation count at which c(n) = 1.0.

    Returns
    -------
    dict
        ss_classical, ss_adjusted, confidence_discount, reduction_pct,
        capital_saved_units.
    """
    variance_component = np.sqrt(
        avg_lead_time * std_demand**2 + avg_demand**2 * std_lead_time**2
    )
    ss_classical = service_level_z * variance_component

    if n_observations is not None:
        c_n = confidence_discount(n_observations, enterprise_benchmark)
    else:
        c_n = 1.0

    ss_adjusted = (service_level_z * c_n) * variance_component

    return {
        'ss_classical': round(ss_classical, 1),
        'ss_adjusted': round(ss_adjusted, 1),
        'confidence_discount': round(c_n, 4),
        'reduction_pct': round((1 - c_n) * 100, 1),
        'capital_saved_units': round(ss_classical - ss_adjusted, 1)
    }


# ============================================================================
# Component 2: Iterative Forecast-Error Correction
# ============================================================================

def rolling_mape(actual: np.ndarray,
                 forecast: np.ndarray,
                 window: int = 30) -> List[Dict]:
    """
    Calculate rolling MAPE and derive a forecast-error correction factor.

    After each forecast period, the resulting MAPE is used to adjust
    the confidence-discount factor c(n), creating a feedback loop
    between forecast performance and inventory policy.

    Parameters
    ----------
    actual : np.ndarray
        Array of actual demand values.
```

```
    forecast : np.ndarray
        Array of forecast demand values.
    window : int, default 30
        Rolling window size for MAPE calculation.

    Returns
    -------
    list of dict
        Each dict contains: period, rolling_mape, mape_adjustment_factor.
    """
    results = []
    for i in range(window, len(actual)):
        a_window = actual[i - window:i]
        f_window = forecast[i - window:i]

        mask = a_window > 0
        if mask.sum() > 0:
            mape = np.mean(np.abs(
                (a_window[mask] - f_window[mask]) / a_window[mask]
            )) * 100
        else:
            mape = 100.0

        # As MAPE decreases (forecast improves), the adjustment factor
        # approaches 1.0, allowing c(n) to increase toward full confidence.
        mape_factor = max(0.8, 1 - mape / 200)

        results.append({
            'period': i,
            'rolling_mape': round(mape, 2),
            'mape_adjustment_factor': round(mape_factor, 4)
        })

    return results


def adjusted_confidence_with_mape(c_n: float, mape_factor: float) -> float:
    """
    Apply MAPE-based correction to the confidence discount.

    c_adjusted = c_n * mape_factor

    This creates the feedback loop: as forecast accuracy improves
    over successive cycles, the safety-stock recommendation becomes
    more precise.
    """
    return round(min(1.0, c_n * mape_factor), 4)


# ============================================================================
# Component 3: Composite Capacity-Risk Score (CCRS)
# ============================================================================

def supplier_concentration_hhi(supplier_shares: pd.Series) -> Tuple[float, float]:
    """
    Calculate the Herfindahl-Hirschman Index for supplier concentration.

    Parameters
    ----------
    supplier_shares : pd.Series
        Normalized value counts of supplier assignments.

    Returns
    -------
    tuple of (disruption_score, hhi)
        disruption_score: 0-100 where higher = less concentrated (less risk).
        hhi: Raw Herfindahl-Hirschman Index (0 to 1).
    """
    hhi = (supplier_shares ** 2).sum()
    score = max(0, min(100, (1 - hhi) * 150))
    return round(score, 1), round(hhi, 4)


def composite_capacity_risk_score(utilization_efficiency: float,
```

```
                                  forecast_reliability: float,
                                  disruption_exposure: float,
                                  w_util: float = 0.40,
                                  w_forecast: float = 0.35,
                                  w_disruption: float = 0.25) -> Dict:
    """
    Calculate the Composite Capacity-Risk Score (CCRS).

    CCRS = w_util * Utilization + w_forecast * Forecast + w_disr * Disruption

    All inputs are normalized scores on a 0-100 scale.

    Parameters
    ----------
    utilization_efficiency : float
        How effectively existing capacity is deployed (0-100).
    forecast_reliability : float
        Inverse of rolling MAPE, normalized to 0-100.
    disruption_exposure : float
        Inverse of supplier concentration risk (0-100).
    w_util, w_forecast, w_disruption : float
        Component weights (must sum to 1.0).

    Returns
    -------
    dict
        ccrs_score, component breakdowns, risk_category.
    """
    assert abs(w_util + w_forecast + w_disruption - 1.0) < 0.001, \
        "Weights must sum to 1.0"

    # Each weighted component is reported to one decimal place, and the score is
    # the sum of those reported components, so the score always equals its parts.
    utilization_component = round(w_util * utilization_efficiency, 1)
    forecast_component = round(w_forecast * forecast_reliability, 1)
    disruption_component = round(w_disruption * disruption_exposure, 1)
    ccrs = round(utilization_component + forecast_component + disruption_component, 1)

    if ccrs >= 70:
        category = 'Low Risk'
    elif ccrs >= 50:
        category = 'Moderate Risk'
    else:
        category = 'High Risk'

    return {
        'ccrs_score': ccrs,
        'utilization_component': utilization_component,
        'forecast_component': forecast_component,
        'disruption_component': disruption_component,
        'risk_category': category
    }


if __name__ == '__main__':
    # Quick self-test with the illustrative example from the methodology doc
    result = safety_stock_adjusted(
        avg_demand=40, std_demand=12,
        avg_lead_time=7, std_lead_time=1.5,
        service_level_z=1.65, n_observations=426  # ~14 months of weekdays
    )
    print("Self-test (Section 4 illustrative example):")
    print(f"  Classical SS: {result['ss_classical']} units")
    print(f"  Adjusted SS:  {result['ss_adjusted']} units")
    print(f"  c(n):         {result['confidence_discount']}")
    print(f"  Reduction:    {result['reduction_pct']}%")
```